ÉTUDE DE CAS 03 / APPLICATION CLIENT

MOUSSA TOURE

Construire un portfolio administrable pour un comédien

CARLOS DA COSTA

ÉTUDE DE CAS APPROFONDIE · ÉDITION REVUE ET AUGMENTÉE · OCTOBRE 2026

React, TypeScript, Firebase, CMS sur mesure, médiathèque, CV, Cloudinary, Cloudflare et confidentialité.

Note de l’auteur

Je présente ce projet comme un exercice de développement pour un besoin réel : permettre à un acteur de disposer d’une vitrine professionnelle et d’en gérer le contenu sans dépendre d’une modification de code à chaque nouveau rôle ou shooting. Le défi était autant éditorial et ergonomique que technique.

Ce dossier s’appuie sur le dépôt public de démonstration moussa-toure-portfolio-code et sa documentation. Ce dépôt est volontairement différent de la version privée et de son contenu de production : il contient des données fictives, sans les photos originales ni le CV réel. Lorsque je parle de la gestion de médias, j’explique les capacités visibles dans le code et leurs conditions, sans affirmer qu’elles sont toutes activées sur le site déployé.

> **Méthode de lecture**
>
> Je présente les raisonnements de conception et les arbitrages techniques à partir des fichiers et tests vérifiables du projet. Les sections distinguent ce qui est **implémenté dans le code**, ce qui a été **validé par des tests**, et ce qui reste **à confirmer sur les services réels**. La première personne décrit mon appropriation et mon analyse du travail ; elle ne prétend pas prouver que chaque ligne de code a été écrite sans assistance.

Sommaire détaillé

01 Comprendre le besoin : un site et un outil de travail 3

02 Architecture de l'application 4

03 Modéliser un portfolio éditorial 5

04 Concevoir une administration utilisable sans coder 6

05 Gérer photos, couvertures, galeries et CV 7

06 Limites de Firebase Spark et choix de stockage 8

07 Signer et sécuriser les opérations médias 9

08 Authentification et règles Firestore 11

09 Expérience publique, design, langue et référencement 12

10 Difficultés documentées et solutions choisies 13

11 Tests et validation : ce qui est réellement démontré 15

12 Déploiement, sauvegarde et conditions de maintenance 17

13 Arbitrages, limites et perspectives 18

14 Concevoir pour une personne qui n'est pas développeuse 19

15 Modèle éditorial : des contenus autonomes et ordonnables 20

16 L'administration comme système de publication contrôlé 21

17 Images et PDF : séparer données, médias et index 22

18 Cloudinary, Worker et R2 : les frontières de sécurité 23

19 Résoudre les incidents de médias et de CV 24

20 Référencement, performance et quatre types de visibilité 25

21 Publication publique, droits et maintenance du projet 26

22 Synthèse de méthode et décisions argumentées 27

23 Défendre les décisions techniques à l'oral 28

24 Annexes : repères de code, glossaire et limites 30

01 Comprendre le besoin : un site et un outil de travail

Le portfolio doit convaincre deux publics

La première audience est celle qui consulte la carrière de l'acteur : directeurs de casting, professionnels, partenaires ou visiteurs. Elle a besoin d'un site clair, rapide, fidèle à son univers visuel, avec des projets, des expériences, des portraits, un showreel et un accès aux coordonnées professionnelles publiées volontairement.

La deuxième audience est le propriétaire du site. Il doit pouvoir ajouter un rôle, masquer un projet, changer une photographie de couverture, mettre à jour un CV et réorganiser les sections sans ouvrir VS Code. C'est cette deuxième exigence qui transforme un site statique en **CMS sur mesure**.

Mes critères de réussite

- Une présentation publique responsive et bilingue français/anglais.
- Une navigation stable, un showreel identifiable, des fiches projets et une galerie adaptée aux portraits.
- Des contenus éditables depuis /admin : projets, rôles, shootings, training, réseaux et rubriques.
- Une distinction claire entre photo de couverture et images secondaires.
- Une gestion du CV séparée des autres photos pour éviter les confusions d'interface.
- Une authentification administrateur et des contrôles d'autorisation côté base et passerelle média.
- Une gestion des médias compatible avec les limites des services disponibles, sans exposer de secrets serveur.
- Une stratégie de publication du code qui ne divulgue ni les photos ni les données réelles de l'acteur.

Pourquoi une solution personnalisée plutôt qu'un CMS générique ?

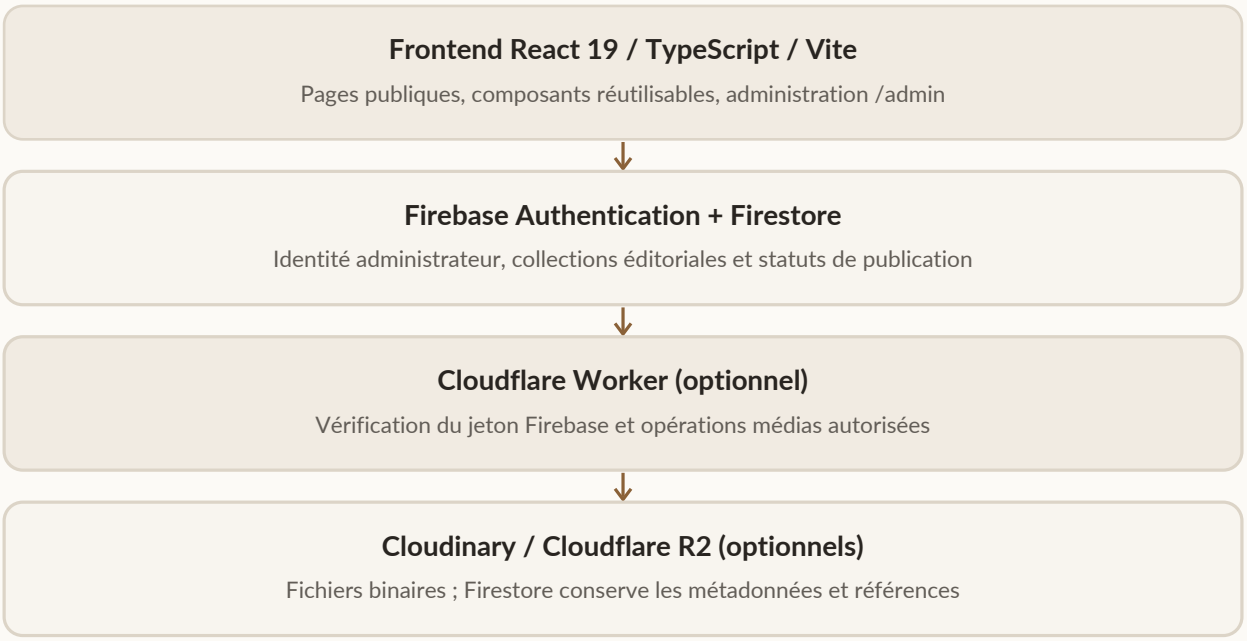
J'ai retenu une interface d'administration intégrée au projet React afin d'adapter le modèle aux besoins spécifiques du portfolio : expériences indépendantes des fiches projets, ordre des sections, choix des couvertures et rubriques artistiques. Cette personnalisation permet des parcours simples pour l'utilisateur, mais augmente ma responsabilité sur les validations, les permissions et la maintenance.

L'objectif n'est pas de reproduire tous les outils d'un CMS universel. Il est de couvrir proprement les actions récurrentes de la personne qui administre ce portfolio.

> **Une règle de conception**

> Pour un utilisateur non développeur, la bonne interface est celle qui lui permet de terminer une modification sans avoir à comprendre le dépôt, la structure Firestore ou la procédure de déploiement.

02 Architecture de l’application



Frontend et routage

Les pages de l’édition publique se trouvent dans **src/pages/** : accueil, administration, pages juridiques et détails de projets ou de récits photographiques. Les éléments transversaux — navigation, galeries, vidéos, shooting, formation ou profil professionnel — sont des composants sous **src/components/**.

La logique de données et d’intégration est isolée dans **src/lib/** : Firebase, localisation, médias, contenus, ordre des sections et connexions aux stockages. Cette organisation rend le produit plus facile à modifier sans mélanger affichage, autorisations et opérations réseau.

Pourquoi TypeScript ?

Le contenu du site ne se limite pas à des paragraphes : une fiche peut posséder une traduction, un statut de publication, une image principale, plusieurs images secondaires, un ordre et des références vers d’autres contenus. Les types de **src/types/models.ts** documentent ces objets et facilitent le repérage d’incohérences à la compilation.

TypeScript n’empêche pas les données Firestore d’être mal formées. Il réduit les erreurs au développement ; les valeurs reçues doivent encore être validées et les règles serveur restent obligatoires.

L’interface, les données, et le stockage ne sont pas une seule chose

Le frontend peut afficher un jeu de données initial, puis utiliser des souscriptions Firestore lorsqu’un environnement Firebase est configuré. Les visiteurs doivent accéder aux contenus publiés, alors que l’administrateur peut modifier les brouillons et les paramètres.

Les médias volumineux sont un autre problème. La base de données se prête aux métadonnées et à la structuration ; un service de stockage d’objets ou de médias est plus adapté aux fichiers binaires.

03 Modéliser un portfolio éditorial

Des collections correspondant au métier

Le site manipule plusieurs familles de contenus : **projects, experiences, shootings, gallery, stories, videos, testimonials, socialLinks, customSections et training**. S’y ajoutent les données de profil et les réglages de site. Cette séparation est volontaire : une expérience de comédien ne correspond pas toujours à un projet audiovisuel complet, et une photo de shooting ne doit pas nécessairement créer une nouvelle fiche de film.

La présence de collections distinctes facilite les affichages spécialisés. Elle rend aussi nécessaire une gestion cohérente des références et de l’ordre d’affichage dans chaque section.

Statuts et publication

Le modèle utilise des statuts comme **draft, published et hidden**. Une fiche en cours ne doit pas être présentée publiquement comme si elle avait été validée. Les règles Firestore conditionnent la lecture des collections éditoriales au statut de publication, avec une exception contrôlée pour l’administrateur.

C’est un élément de sécurité mais aussi de qualité éditoriale : l’acteur peut préparer une information, la relire, puis la publier sans bricoler directement la base et sans redéployer tout le frontend.

Deux langues, une donnée structurée

Les champs bilingues sont modélisés comme des valeurs localisées. La fonction **t** dans src/lib/content.ts choisit la langue et prévoit une valeur de repli. Le composant de contexte **usePortfolio.tsx** conserve l’état de langue.

La difficulté dépasse le simple remplacement des intitulés de menus : titres, biographies, libellés de projets, descriptions et messages d'erreur doivent rester cohérents. Une traduction absente ne doit pas casser la page.

Liens et ressources externes

Les fonctions **safeUrl** et **safeYoutubeId** donnent des garde-fous pour les ressources publiées. Les entrées éditoriales sont traitées comme des données externes même lorsqu'elles viennent d'un formulaire d'administration.

J'en tire un principe : contrôler la forme d'une URL avant de l'afficher évite qu'une fiche ou un champ mal saisi ait des conséquences indésirables sur la navigation ou le chargement de médias.

04 Concevoir une administration utilisable sans coder

Le vrai test est le parcours de l'acteur

Une page /admin n'est utile que si un utilisateur peut reconnaître l'action à effectuer. Le composant **src/pages/Admin.tsx** organise la gestion des collections, les formulaires, les choix de médias, le profil, les textes et la navigation. Il prévoit des opérations comme créer, modifier, réordonner, masquer ou supprimer des fiches.

Je cherche à rendre explicite la différence entre **« enregistrer un contenu »**, **« choisir un média »** et **« publier »**. Mélanger ces actions pourrait supprimer des données ou exposer une photographie avant validation.

Éviter les erreurs de manipulation

Les formulaires suivent la présence de modifications non enregistrées et peuvent demander une confirmation lorsque l'utilisateur quitte un écran. Les opérations asynchrones exposent un état d'attente et une explication d'erreur.

C'est un détail important pour un utilisateur réel : une action longue liée au réseau ne doit pas sembler n'avoir rien fait, et un clic accidentel sur une autre rubrique ne doit pas effacer silencieusement une saisie.

Gérer les expériences indépendamment des projets

Une évolution documentée a consisté à isoler les **expériences et rôles** des fiches projets. L'interface permet d'importer les crédits existants, puis de les gérer comme des éléments éditoriaux autonomes.

La différence de modèle répond à un besoin métier : un crédit de comédien peut être pertinent sans disposer de photographies, d'une fiche complète ou d'un album propre. La conception s'adapte ainsi à la représentation de la carrière, plutôt que de forcer chaque rôle dans la catégorie « projet ».

Ordre des sections

Le module **sectionOrder.ts** complète un ordre enregistré avec les sections disponibles et permet de retrouver des ancres cohérentes. Une nouvelle rubrique ne doit pas disparaître simplement parce qu'elle n'existait pas au moment où l'ordre précédent a été sauvegardé.

Le composant de gestion de layout offre une manière de déplacer les sections. C'est une fonctionnalité éditoriale, mais aussi une décision de compatibilité des données de configuration.

05 Gérer photos, couvertures, galeries et CV

Différencier une couverture d'une galerie

Une fiche doit pouvoir présenter une image principale et plusieurs photographies secondaires. L'administration contient un éditeur de galerie et couverture ; choisir une nouvelle couverture, ajouter des médias et modifier leur ordre sont des opérations différentes.

Dans un modèle de données, l'image principale peut être une propriété spécifique tandis que les autres images forment un tableau ordonné. L'interface doit préserver l'intention de l'utilisateur lors d'un réordonnancement et ne pas recréer des références inutiles.

Pourquoi une médiathèque ?

Si chaque projet porte seulement une URL libre, les mêmes photos sont difficiles à retrouver, remplacer ou supprimer proprement. Une collection **mediaIndex** permet de stocker des références, tailles, types MIME, fournisseurs, identifiants distants et états.

Ce mécanisme facilite le choix d'une photo déjà envoyée et la gestion des doublons. Il impose en échange des procédures de nettoyage : supprimer un objet distant sans vérifier ses usages peut casser plusieurs fiches du site.

Un CV n'est pas une photographie

Un problème d'ergonomie documenté concernait une option de mise à jour du CV cachée parmi les réglages avancés et les outils de photos. Une section distincte de gestion du CV évite que l'utilisateur cherche un PDF dans une galerie d'images.

Le CV a aussi ses propres contraintes : type application/pdf, lien de consultation ou de téléchargement, fichier de remplacement, contrôle de taille, et maintien d'un lien valide après

changement. Dans cette application, le CV peut être géré avec un fournisseur de média configuré ; aucun vrai CV n'est inclus dans le dépôt public de démonstration.

Le problème des contenus déjà publiés

Remplacer un média sans perdre l'ancien contenu demande de distinguer **référence enregistrée** et **fichier réellement présent chez le fournisseur**. Le code contient des opérations de recherche de PDF déjà envoyé, aussi bien côté Cloudinary que R2.

C'est une forme de récupération après incident : un upload peut réussir avant qu'une écriture Firestore échoue. Le système doit alors pouvoir retrouver l'objet distant plutôt que forcer l'utilisateur à le réimporter.

> **Un apprentissage de modélisation**
>
> Le média est un objet technique ; sa place dans une fiche est une décision éditoriale. Séparer ces deux notions permet de changer la couverture, réordonner une galerie et conserver l'historique utile sans déplacer physiquement tous les fichiers.

06 Limites de Firebase Spark et choix de stockage

Pourquoi ne pas placer toutes les photos dans Firestore ?

Firestore limite la taille d'un document. Le code contient une voie destinée aux petites ressources optimisées, avec une limite de l'ordre de **600 Ko de données brutes** et des contrôles de taille pour l'encodage. Cette méthode peut rendre service dans un environnement contraint, mais ne convient pas à une photothèque de haute résolution.

L'intégration de nombreuses photographies ou d'un CV volumineux nécessite un stockage adapté aux objets binaires et à leur diffusion. Le besoin exprimé de gérer des fichiers importants a donc conduit à étudier des fournisseurs externes.

Trois mécanismes coexistants dans le code

Voie	Utilisation et limite
Firestore mediaAssets	Petites ressources optimisées ; limites strictes des documents.
Cloudinary	Images et PDF avec upload signé, URL de diffusion et métadonnées Firestore ; activation optionnelle.
Cloudflare R2	Fichiers objets avec upload multipart et passerelle Worker ; activation optionnelle.

Pourquoi Cloudinary ?

Cloudinary répond au besoin de stocker et diffuser des images, sans transférer la clé privée dans le navigateur. Le frontend demande une opération à une passerelle qui vérifie l'identité administrateur et signe la requête autorisée.

Les règles Firestore n'enregistrent que les métadonnées et bornent les formats et tailles. Le code distingue un plafond d'image Cloudinary autour de 25 Mo et une limite PDF CV autour de 10 Mo dans les règles de l'édition analysée. Ces valeurs sont des contraintes de validation technique, pas une promesse de quota commercial illimité.

Pourquoi prévoir R2 en option ?

Cloudflare R2 apporte une autre possibilité de stockage objet. Le code implémente un transfert multipart, qui évite d'envoyer un fichier volumineux en une seule requête Worker. Les morceaux ont une taille bornée ; le client réessaie une partie échouée avant d'abandonner proprement.

Le fournisseur reste optionnel. Sa présence dans le code ne signifie ni qu'il est activé en production, ni qu'il supprime toute contrainte de prix, de bande passante, de taille ou de sécurité. Les paramètres réels et les quotas doivent être vérifiés au moment de l'activation.

07 Signer et sécuriser les opérations médias

Pourquoi la signature ne peut pas être faite dans React

La clé privée servant à signer un envoi Cloudinary ne doit jamais être intégrée à un bundle Vite ou à une variable VITE_*. Toute information embarquée dans l'application navigateur peut être lue par un visiteur.

Le **cloudinary-worker/index.js** reçoit une demande accompagnée d'un jeton Firebase, vérifie les autorisations, puis exécute les opérations médias compatibles avec la configuration du serveur.

Vérifier un jeton Firebase de bout en bout

Le Worker ne se contente pas de décoder le contenu d'un JWT : il vérifie l'algorithme RS256, l'émetteur, l'audience correspondant au projet Firebase, les échéances temporelles, l'identité, l'adresse e-mail vérifiée et le **custom claim admin**. Il récupère également les clés publiques JWKS et valide cryptographiquement la signature.

Cela illustre une différence importante entre **« le token affirme que je suis admin »** et **« un serveur de confiance a effectivement signé une preuve valide pour ce projet »**.

Vérifications sur le fichier lui-même

Pour R2, le code contrôle les types autorisés (JPEG, PNG, WebP, AVIF, PDF), la taille annoncée et une signature de format en début de fichier. Une extension .pdf ne suffit pas à prouver que les octets représentent un PDF.

Le Worker contrôle aussi les clés d'objets, les numéros de morceaux, la taille et la confirmation finale. Une origine CORS autorisée est requise pour les opérations d'administration ; cela s'ajoute à l'authentification, sans la remplacer.

```
Admin se connecte à Firebase
-> obtient un ID token signé
Frontend demande /sign ou /r2/create au Worker
-> contrôle Origin + Bearer token + claim admin
-> valide type, taille et destination
Cloudinary / R2 reçoit le fichier
-> référence et métadonnées enregistrées dans Firestore
Le site public utilise seulement une URL de diffusion
```

Que faire si l'upload et l'index divergent ?

Le code R2 distingue un transfert terminé d'un index Firestore enregistré. S'il a bien envoyé le fichier mais n'a pas pu enregistrer sa référence, il signale explicitement à l'administrateur de conserver la clé de récupération plutôt que de relancer aveuglément l'envoi.

Cette gestion des erreurs partielles est une difficulté classique des architectures distribuées : le stockage binaire et la base de métadonnées ne participent pas à une transaction unique.

08 Authentification et règles Firestore

Une page /admin n'accorde aucun droit

Le fait de pouvoir ouvrir l'URL /admin ou d'observer des composants d'édition ne doit pas suffire à modifier les données. Le système utilise Firebase Authentication et un claim administrateur attribué par une procédure privilégiée. Les règles Firestore exigent à la fois l'identité connectée, le claim et la vérification de l'e-mail.

La fonction **isAdmin()** du fichier firestore.rules montre ce principe directement. Les lectures publiques concernent les fiches publiées ; les écritures restent réservées à l'administrateur autorisé.

Défense en profondeur

Je distingue trois contrôles complémentaires : le frontend masque ou présente les actions selon le contexte, Firestore protège l'écriture des documents, et le Worker contrôle les opérations sur les médias.

Un utilisateur malveillant peut modifier son navigateur, fabriquer un appel API ou changer un identifiant de document ; le serveur ne doit donc pas déduire ses permissions de l'état de l'interface.

Conserver des secrets hors du dépôt

Les fichiers d'exemple comme **.env.example** et **.firebaserc.example** documentent la configuration sans publier les véritables secrets ou paramètres de gestion privés. Les scripts d'attribution de droits Firebase exigent un environnement authentifié et un argument de confirmation.

Une clé API Firebase web ne remplace pas un secret de serveur ; les secrets Cloudinary et les accès R2 restent dans les variables protégées du Worker. La documentation déconseille aussi de versionner les caches Wranger, les certificats et les exports de base.

Statuts de contenu et médias publics

Le statut published permet une lecture publique selon les règles, mais la confidentialité d'un fichier ne dépend pas seulement de Firestore. Une URL publique de média reste accessible à toute personne qui la possède.

Avant de publier une photo, il faut vérifier le droit à l'image, l'accord de diffusion, les métadonnées éventuelles et la présence d'informations de contact. Une galerie publique n'est pas un stockage privé.

09 Expérience publique, design, langue et référencement

Une navigation stable sur plusieurs écrans

Un portfolio est souvent consulté depuis un téléphone. Le code comporte un header fixe responsive, des sections adaptées aux différentes tailles et un ordre de navigation synchronisé avec la configuration administrateur.

La navigation doit rester cohérente même lorsqu'une section est masquée ou réordonnée. Cela impose de séparer les clés techniques des libellés affichés et de conserver des ancres prévisibles.

Photographies et fiches en contexte

Les galeries, shootings et fiches de projets ont des besoins de mise en page différents. Une photo de portrait doit respecter son cadrage, alors qu'une image de couverture est présentée dans un contexte plus large. Le composant MediaImage permet de résoudre la référence avant l'affichage.

Cette couche évite que chaque page réimplémente sa propre logique de récupération de médias et facilite le comportement de repli lorsqu'un lien n'est pas disponible.

Showreel, vidéos et liens externes

Le showreel constitue une rubrique indépendante ; les vidéos peuvent utiliser des identifiants validés plutôt que des liens arbitraires. Les composants de vidéo et de récits photographiques séparent l'identité du contenu de son mode d'affichage.

Le risque technique est de dégrader les performances et la confidentialité avec des intégrations tierces trop lourdes. Le projet comprend aussi une gestion du consentement Analytics lorsqu'il est configuré.

SEO : la différence entre une application SPA et une page indexable

Une application React peut afficher correctement une fiche dans le navigateur sans fournir immédiatement au moteur de recherche un HTML complet pour cette fiche. Le dépôt comprend des scripts de pré-rendu et d'export SEO.

La conséquence est un compromis : des données Firestore peuvent être mises à jour depuis /admin sans redéployer, mais le contenu HTML pré-rendu peut nécessiter un nouvel export et un build pour refléter les dernières modifications. Je dois expliquer cette limite au propriétaire du site.

10 Difficultés documentées et solutions choisies

L’acteur ne pouvait pas gérer facilement les contenus

Contexte et problème. Une nouvelle expérience nécessitait parfois un changement de code ou une organisation de données complexe.

Analyse. Un portfolio pensé uniquement pour son développeur ne répond pas au besoin de mise à jour autonome.

Réponse technique. Création de formulaires et collections administrables, avec statuts, ordre et avertissements de modification non enregistrée.

Vérification. Tests du tableau /admin et de la logique des collections dans la suite publique.

Ce que je retiens. L’usage réel doit être validé régulièrement avec l’administrateur.

Toutes les photos aboutissaient à une catégorie imprévue

Contexte et problème. Un parcours de galerie faisait apparaître une catégorisation non souhaitée.

Analyse. Un champ de catégorie mal exposé ou une valeur par défaut implicite rendait le résultat difficile à maîtriser.

Réponse technique. L’éditeur de contenu et les champs de classification doivent rendre le choix explicite et conserver la valeur sélectionnée.

Vérification. Vérifier les formulaires, l’écriture Firestore et l’affichage dans la catégorie choisie.

Ce que je retiens. Ce point est un problème historiquement signalé ; le dépôt public ne prouve pas à lui seul sa résolution complète en production.

La gestion du CV était difficile à trouver

Contexte et problème. Le changement de CV était associé à des outils de photos ou enfoui dans des options avancées.

Analyse. Le besoin métier d’un document professionnel a un sens distinct de l’édition de galeries.

Réponse technique. Une section de gestion de CV dédiée, distincte des photos, avec vérification PDF et possibilité de retrouver une référence existante.

Vérification. Tests admin-photos-cv-split, cv-cloudinary et cv-recovery.

Ce que je retiens. Le vrai CV et les conditions de diffusion ne font pas partie du dépôt public.

Firestore ne convenait pas aux grandes photographies

Contexte et problème. La limite de document et les coûts de transfert rendaient une grande médiathèque difficile.

Analyse. Une base de documents n'est pas un stockage d'objets haute résolution.

Réponse technique. Séparation des métadonnées et des binaires ; voies Cloudinary et R2 optionnelles, tout en gardant le mécanisme d'images optimisées.

Vérification. Tests cloudinary-media et r2-storage, contrôles de type et de taille.

Ce que je retiens. L'activation des fournisseurs, leurs quotas et leur coût relèvent d'une vérification réelle distincte.

Un envoi pouvait réussir avant l'enregistrement Firestore

Contexte et problème. Une opération se répartit entre deux services sans transaction commune.

Analyse. Un échec d'écriture d'index après upload peut laisser un objet orphelin.

Réponse technique. Messages d'erreur contenant la référence à conserver et fonctions de récupération pour Cloudinary/R2.

Vérification. Tests de récupération et contrôle des métadonnées.

Ce que je retiens. Une procédure régulière de recherche d'objets orphelins resterait utile.

Les permissions admin ne doivent pas être usurpables

Contexte et problème. Une URL /admin connue ou un JWT modifié ne doit pas permettre d'envoyer un média.

Analyse. Le frontend et le simple décodage d'un jeton ne suffisent pas.

Réponse technique. Règles Firestore avec claim admin et e-mail vérifié ; Worker vérifiant audience, émetteur, signature RS256 et durée de validité.

Vérification. Tests des règles et de la configuration, examen du Worker et des contrôles d'en-tête.

Ce que je retiens. La configuration effectivement déployée et la rotation des secrets doivent encore être vérifiées.

L'ordre de navigation pouvait perdre de nouvelles rubriques

Contexte et problème. Une ancienne préférence de menu ne mentionne pas les sections ajoutées ensuite.

Analyse. Une liste persistée ne doit pas être traitée comme la liste absolue des fonctionnalités disponibles.

Réponse technique. completeSectionOrder fusionne l'ordre sauvegardé et les sections autorisées, sans supprimer les nouvelles.

Vérification. Tests training-legacy-order et training-navigation.

Ce que je retiens. Les ancres et le responsive doivent être testés après chaque ajout de rubrique.

Publier le dépôt original exposerait des médias historiques

Contexte et problème. Le dépôt privé contenait des photographies et un CV, y compris dans des commits antérieurs.

Analyse. Supprimer les fichiers du dernier commit ne retire pas leur présence de l'historique Git.

Réponse technique. Création d'un dépôt public indépendant avec nouveau premier commit, médias supprimés et contenus fictifs.

Vérification. Audit de publication : retrait de 143 images et d'un PDF, scanner de secrets et tests publics.

Ce que je retiens. La séparation ne remplace pas une revue juridique des droits sur le code lui-même.

11 Tests et validation : ce qui est réellement démontré

Une suite publique volontairement différente

Le dépôt public conserve une suite de **38 tests réussis sur 38** lors de la préparation de sa version nettoyée. Ce nombre est propre à la copie publiable : certains tests dépendant des photographies, du CV ou des données réelles ont été retirés. Les contrôles GitHub Actions de la branche main ont également réussi.

Le workflow public exécute **npm ci**, les tests Node, le typecheck TypeScript et une recherche de secrets. Il ne démontre pas automatiquement le bon fonctionnement du site de production, d'un vrai compte administrateur ou du stockage cloud configuré.

Famille de tests	Objet du contrôle
Administration	Composants d'édition, séparation CV/photos et parcours de navigation.
Médias	Envoi et références Cloudinary/R2, format des fichiers, récupération de CV.

Famille de tests	Objet du contrôle
Configuration	Absence de dépendance aux secrets de production dans le dépôt public.
Internationalisation	Choix de langue et cohérence de traductions.
Navigation	Ordre ancien, training, header et emplacement des sections.
Publication	Vérification que les contenus privés et historiques ne sont pas inclus.
TypeScript	Cohérence statique des contrats utilisés par React et les modules.

Un vrai plan de test utilisateur

- Ouvrir le site en navigation privée : seules les fiches publiées doivent apparaître.
- Se connecter avec un compte non administrateur et vérifier que les écritures Firestore et les routes Worker sont refusées.
- Se connecter avec le compte admin et ajouter une fiche de démonstration, puis la masquer et la réordonner.
- Téléverser une petite image autorisée, puis changer la couverture et l'ordre de la galerie.
- Tester un upload échoué, une récupération de PDF et l'absence de doublon après erreur.
- Vérifier en français et en anglais les fiches, textes de navigation et messages d'erreur.
- Contrôler sur téléphone et ordinateur le showreel, le header, le focus clavier et les liens.
- Réaliser une sauvegarde avant toute manipulation de données réelles ou déploiement de règles.

Les vérifications qu'il manque encore

Le rapport de publication rappelle que la version publique n'avait pas validé son build Vite complet dans l'environnement de préparation. Le workflow GitHub observé ne contient pas non plus cette étape. J'éviterais donc d'annoncer un pipeline de compilation entièrement certifié sans exécuter explicitement **npm run build**.

Les règles de production, les services Cloudinary/R2 et le comportement d'un compte administrateur réel exigent également des essais complémentaires. Un site actuellement disponible en ligne ne prouve pas que toutes les fonctionnalités optionnelles du dépôt public y sont activées.

12 Déploiement, sauvegarde et conditions de maintenance

Deux flux de changement différents

Une modification de contenu publiée dans Firestore peut devenir visible immédiatement, tandis qu'une modification de code React nécessite un build et un déploiement Hosting. La configuration des règles ou du Worker constitue encore un troisième type d'intervention, avec des conséquences de sécurité et de disponibilité.

Je sépare donc **éditer une fiche**, **publier une nouvelle interface** et **changer l'infrastructure**. Cette distinction évite de faire exécuter à l'utilisateur une procédure technique pour une action éditoriale courante.

Pourquoi sauvegarder avant de modifier les règles ?

Les règles Firestore décrivent les droits, pas simplement des préférences. Les écraser sans comparaison peut bloquer l'administrateur ou ouvrir accidentellement des collections. Les guides du dépôt recommandent de sauvegarder l'environnement actuel, d'utiliser un projet de test et de relire les modifications avant déploiement.

La même prudence s'applique aux scripts d'import ou d'attribution de claim admin. Un script qui peut écrire dans la base doit exiger un environnement explicite et une confirmation compréhensible.

Mise en ligne et référencement

Le frontend utilise Vite et Firebase Hosting lorsque l'environnement est configuré. Le build produit un dossier dist ; les scripts de pré-rendu peuvent nécessiter d'autres étapes pour mettre à jour les pages indexables.

Une modification de galerie dans /admin ne doit pas entraîner automatiquement une migration des règles ou un redéploiement d'un Worker. Je privilégie des changements ciblés, accompagnés de tests et d'une possibilité de retour arrière.

Le dépôt public comme objet distinct

La version publiable a été préparée avec des données de démonstration, un nouvel historique Git et des fichiers de configuration non secrets. Elle ne doit pas être utilisée pour redéployer directement la production.

Cette séparation fournit un support d'étude à un lecteur extérieur tout en diminuant le risque de diffuser les photos, les témoignages et le CV d'une autre personne. Le code source peut être public sans que tous les contenus associés le soient.

13 Arbitrages, limites et perspectives

Les qualités que je peux montrer

- Une réponse à un besoin réel de gestion de carrière artistique, pas seulement une page vitrine.
- Une séparation claire entre données éditoriales, identité administrateur et fichiers média.
- Une administration conçue pour être utilisée sans modifier le code source.
- Des règles de publication, des contrôles d'autorisation et une passerelle sécurisée de signature.
- Une réflexion sur les limites de Firestore et les options de stockage externe.
- Une stratégie de dépôt public nettoyé respectant la confidentialité et le droit à l'image.

Le coût de la flexibilité

Proposer plusieurs fournisseurs de médias facilite la résilience et l'évolution, mais multiplie les chemins de code et les cas d'erreur. Il devient important de documenter quel fournisseur est réellement activé, quels formats sont supportés et comment récupérer un fichier perdu de vue.

Un CMS sur mesure ajoute aussi une responsabilité d'accessibilité, de sauvegarde et de compatibilité des modèles lorsque l'interface évolue. Les tests unitaires de composants ne suffisent pas à évaluer l'expérience d'un administrateur qui utilise le site occasionnellement.

Ma feuille de route

Priorité	Amélioration mesurable
1	Inclure npm run build dans la CI de la version publique et vérifier le résultat.
2	Tester le refus des opérations admin avec un compte ordinaire et un jeton invalide.
3	Formaliser un registre des médias utilisés et une procédure de récupération des objets orphelins.
4	Tester les parcours d'administration avec l'acteur sur mobile et ordinateur.
5	Réviser le référencement après une mise à jour Firestore : comportement public et pages prérendues.
6	Contrôler régulièrement autorisations des médias, confidentialité et durée de conservation.

Ce que j'ai appris

Ce projet m'a surtout confronté à une différence fondamentale entre développer pour soi et développer pour une autre personne : les décisions techniques doivent se traduire par des parcours compréhensibles, sûrs et maintenables.

Le meilleur indicateur n'est pas le nombre de technologies utilisées, mais la possibilité pour l'acteur de mettre son portfolio à jour sans risque ni dépendance excessive, tandis que je peux expliquer comment les permissions, le stockage et les tests soutiennent cette autonomie.

14 Concevoir pour une personne qui n'est pas développeuse

Le projet répond à deux utilisateurs différents : le visiteur qui consulte une vitrine artistique et l'acteur qui doit la tenir à jour sans modifier de composants React.

Les tâches éditoriales comme point de départ

Le besoin principal peut se formuler sans vocabulaire informatique : publier une expérience, choisir une photo de couverture, présenter plusieurs images d'un shooting, remplacer un CV, masquer provisoirement une fiche ou modifier l'ordre d'apparition des rubriques. Une interface /admin utile traduit ces gestes en formulaires compréhensibles et en états d'enregistrement explicites.

J'ai intérêt à raisonner en charge cognitive plutôt qu'en nombre de paramètres. Un acteur qui doit retrouver le contrôle du CV dans un menu avancé risque de ne jamais mettre le document à jour. Séparer la gestion des images de celle du CV constitue donc une décision ergonomique autant que technique : un PDF professionnel possède des contraintes de format, de visibilité et de récupération différentes d'une photographie.

Mes critères de réussite

Tâche	Critère observable	Risque si mal conçu
Ajouter un rôle	Sans création artificielle de projet film	Modèle rigide
Changer la couverture	Image principale choisie et galerie préservée	Perte d'ordre ou de contexte
Mettre le CV à jour	Section visible, PDF validé	Document périmé ou introuvable
Masquer une fiche	N'apparaît plus publiquement	Confidentialité insuffisante
Réordonner les sections	Menu et page affichent le même ordre	Navigation contradictoire

Un projet client comme exercice de communication

Je distingue ce que l'utilisateur demande de la solution technique que je choisis. « Je veux ajouter plusieurs photos » ne veut pas dire « il faut les stocker dans Firestore ». L'exigence fonctionnelle concerne la publication, l'ordre, la récupération et le remplacement. C'est ensuite au développeur de comparer les technologies et de choisir un stockage cohérent. Cette séparation me permet de justifier des choix sans imposer à l'utilisateur un jargon inutile.

15 Modèle éditorial : des contenus autonomes et ordonnables

Différencier projets, crédits, shootings et formation

Une fiche de projet audiovisuel, une expérience professionnelle et un shooting ne représentent pas la même chose. Une expérience peut être une ligne de crédit, parfois sans images ; un shooting privilégie des médias ; un projet peut posséder une page détaillée, une description et plusieurs catégories. Les traiter comme des collections ou types distincts évite de forcer l'acteur à créer de faux projets pour renseigner un simple rôle.

src/types/models.ts établit les formes TypeScript nécessaires à cette diversité. L'application publique utilise des données fictives initiales, tandis que les contenus effectivement publiés peuvent être synchronisés avec Firestore. Ce modèle rend possible une interface riche tout en conservant une organisation de données intelligible.

La compatibilité des anciennes préférences

src/lib/sectionOrder.ts apporte un exemple particulièrement révélateur. Lorsqu'une nouvelle rubrique apparaît, la préférence d'ordre enregistrée par un utilisateur existant ne la contient pas forcément. Remplacer sans nuance cet ordre détruirait ses choix ; conserver la liste sans adaptation ferait disparaître la nouvelle rubrique. completeSectionOrder conserve les clés encore autorisées, évite les doublons puis insère celles qui manquent à un emplacement cohérent.

Ce mécanisme transforme une évolution du modèle en migration progressive côté application, sans exiger immédiatement une réécriture de tous les documents Firestore. Les tests training-legacy-order et training-navigation documentent ce cas de compatibilité.

Cas	Décision de modèle	Effet sur l'interface
Projet vidéo	Fiche et médias associés	Page détaillée
Crédit professionnel	Expérience indépendante	Liste de rôles
Shooting	Galerie et couverture	Présentation photo

Cas	Décision de modèle	Effet sur l'interface
Formation	Entrée structurée	Rubrique dédiée
Ordre de section ancien	Compléter les nouvelles clés	Navigation conservée

16 L'administration comme système de publication contrôlé

Statuts et visibilité

La publication ne se réduit pas au fait qu'un document existe. Une fiche peut être en brouillon, publiée ou masquée. Le frontend public utilise une requête filtrée sur le statut published pour de nombreuses collections. Les règles Firestore distinguent l'accès public à ces objets et les droits d'écriture des administrateurs. Un simple champ « masqué » dans React ne constituerait pas une protection de données.

src/lib/content.ts crée une représentation initiale des contenus puis ouvre des souscriptions onSnapshot lorsque Firebase est disponible. Cette organisation offre une démonstration locale, mais l'édition réelle des contenus repose sur des services correctement configurés. La présence d'une donnée de démonstration ne doit pas être présentée comme un contenu réel de l'acteur.

Une modification doit traverser plusieurs contrôles

Le parcours de publication part de la saisie dans /admin, puis de la validation du type et de l'état, passe par l'authentification Firebase et aboutit à une écriture autorisée par Firestore. Les visiteurs voient ensuite les contenus publiés à partir des requêtes disponibles. Le système reste cohérent seulement si l'UI, les règles et les données utilisent les mêmes statuts et les mêmes champs.

Une erreur de sauvegarde ne doit pas laisser croire que les changements sont publiés. Je privilégie donc les validations de champs, les états de chargement, les avertissements de modifications non enregistrées et les réponses d'erreur compréhensibles. Le bon critère est qu'une personne occasionnelle puisse répéter le geste sans assistance technique.

Opération	UI	Autorisation attendue
Créer une fiche	Formulaire et aperçu	Compte admin vérifié
Changer une photo	Choix de couverture	Écriture contrôlée et média valide
Publier	Passage au statut published	Règle Firestore appropriée

Opération	UI	Autorisation attendue
Masquer	Statut hidden	Invisibilité dans la requête publique
Réordonner	Modification de order	Aucune clé de navigation perdue

17 Images et PDF : séparer données, médias et index

Pourquoi une base documentaire ne suffit pas

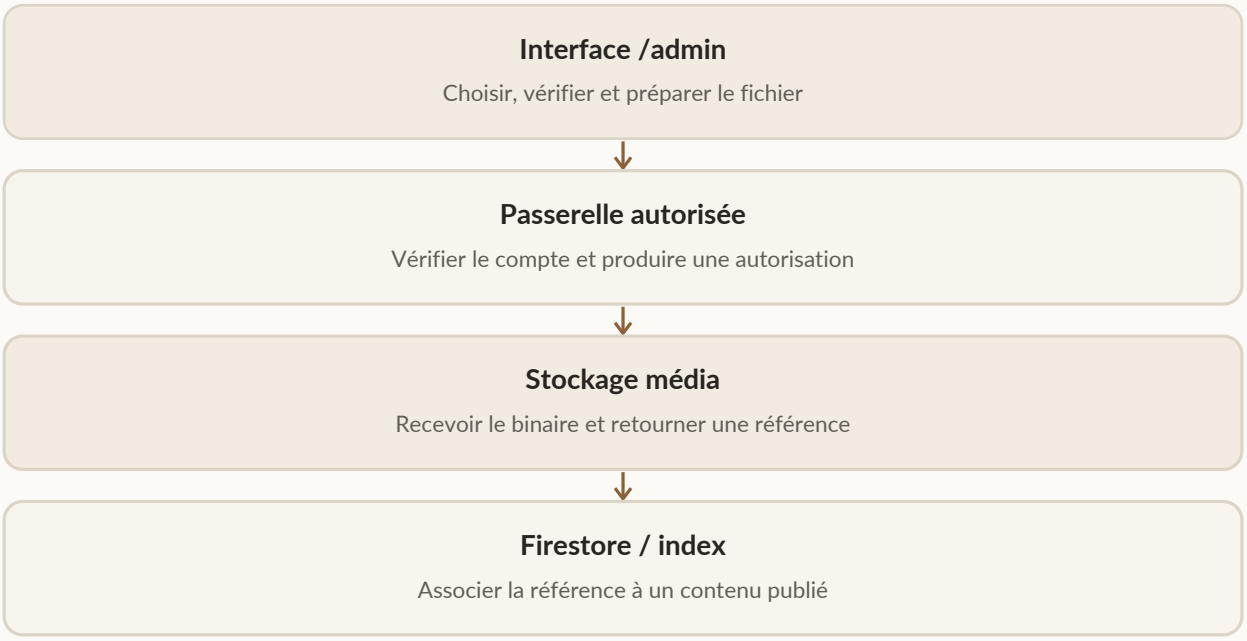
Firestore convient aux documents de métadonnées : titre, catégorie, ordre, référence de média, état de publication. Le stockage de binaires volumineux est un autre métier, avec des contraintes de taille, de coût de transfert, de bande passante et de diffusion. Le projet expose donc des mécanismes d'upload et de récupération utilisant des services spécialisés lorsque ceux-ci sont configurés.

src/lib/media.tsx conserve également une logique de préparation et de restitution de fichiers adaptés au frontend. Le fait qu'un service média existe n'interdit pas l'optimisation client, le cache ou la validation des types. Au contraire, chaque couche doit contribuer à éviter un upload disproportionné et à améliorer l'expérience de consultation.

Trois objets différents derrière une photo

Je distingue le fichier binaire, sa référence technique chez le fournisseur et le document éditorial qui l'associe à une fiche. Cette décomposition est importante pour la récupération : retrouver un fichier chez Cloudinary ou R2 ne signifie pas automatiquement qu'il est indexé et visible dans la galerie. Elle explique aussi pourquoi supprimer une fiche et supprimer définitivement son fichier ne devraient pas être des opérations confondues.

Dans les parcours les plus sensibles, une tentative d'upload peut réussir alors que l'écriture des métadonnées échoue. Il n'existe pas de transaction commune entre une base Firestore et un fournisseur externe. La solution documentée consiste à conserver un identifiant de récupération afin de reprendre l'indexation sans renvoyer aveuglément le même fichier.



18 Cloudinary, Worker et R2 : les frontières de sécurité

La passerelle ne remplace pas l'identité

Le code cloudinary-worker/index.js s'appuie sur un jeton Firebase vérifié cryptographiquement. Il contrôle notamment la signature RS256, l'émetteur, l'audience et la validité temporelle avant d'autoriser les routes administratives. Le claim d'administration et la vérification de l'adresse e-mail font partie des préconditions. Cette architecture évite de donner au navigateur les secrets privés du fournisseur média.

Une liste d'origines autorisées est une protection complémentaire, mais CORS n'est pas une authentification. Un attaquant capable d'envoyer une requête hors navigateur n'est pas arrêté par la seule présence d'un en-tête Origin. Je peux justifier l'architecture en montrant que les permissions sont vérifiées par le token et que l'origine restreint en plus le contexte web accepté.

Un protocole multipart contrôlé

src/lib/r2.ts décrit un upload optionnel qui ne devient actif que si la configuration explicite est présente. Les fichiers autorisés comprennent les images JPEG, PNG, WebP, AVIF et les PDF. Le code limite volontairement la taille à 500 Mo, vérifie les signatures de format des premiers octets et prépare des morceaux de 6 Mo pour l'upload multipart. Ces paramètres sont des limites applicatives ; ils ne constituent pas la promesse d'un stockage gratuit ou illimité.

La route d'envoi retourne pour chaque morceau un numéro et une empreinte ETag attendue. L'application valide cette réponse avant de poursuivre. Une clé média suit un schéma fermé, ce qui empêche le client de fournir librement un chemin arbitraire à modifier. Cette approche réduit la surface d'attaque, mais doit toujours être accompagnée de quotas, d'autorisations serveur et d'un contrôle des coûts chez le fournisseur.

Contrôle	Exemple concret	Finalité
Utilisateur	Firebase ID token signé	Réserver les actions au compte admin
Origine	Domaine autorisé	Limiter les contextes navigateur
Type de fichier	MIME et signature binaire	Refuser les faux PDF et images
Taille	Limite applicative de 500 Mo	Maîtriser le volume maximum
Clé R2	Motif fermé par expression régulière	Éviter les chemins arbitraires
Multipart	Numéro de morceau et ETag	Détecter une confirmation inattendue

19 Résoudre les incidents de médias et de CV

Un document peut être présent sans être retrouvable

Le problème d'un CV envoyé puis introuvable révèle un défaut de séparation des étapes : l'upload peut avoir abouti, alors que sa référence n'a pas été enregistrée ou affichée. src/lib/cloudinary.ts propose lookupCloudPdf pour retrouver un PDF connu, et les tests cv-recovery.test.mjs examinent la restriction aux origines autorisées, l'authentification Firebase et le refus d'identifiants arbitraires.

Ce scénario m'a conduit à privilégier une réparation non destructive. Réenvoyer le document avant de chercher la version existante multiplie les fichiers et obscurcit l'historique. Conserver la référence, vérifier ses métadonnées et reprendre l'écriture de l'index produit un chemin de récupération plus maîtrisé.

Séparer physiquement la gestion des photos et du CV

Une section d'administration unique pouvait faire du changement de CV un détail perdu dans la gestion des photographies. Le test admin-photos-cv-split.test.mjs aide à vérifier que l'interface distingue ces opérations. Cette amélioration illustre une méthode : corriger un problème d'ergonomie identifié par l'utilisateur, puis inscrire le résultat attendu dans un test de non-régression.

La même discipline s'applique aux galeries : un problème de catégories peut provenir d'une valeur par défaut, d'un champ masqué ou d'une transformation à l'écriture. Il est nécessaire de suivre la donnée depuis le formulaire jusqu'au document Firestore puis à la requête publique, sans annoncer qu'un problème historique est définitivement réglé si la version de production n'a pas été revalidée.

Séquence de récupération du CV

Contexte et problème. Le fichier a potentiellement atteint Cloudinary, mais le document de liaison n'a pas été conservé correctement.

Analyse. Le stockage externe et Firestore ne partagent pas de transaction ; une perte partielle est donc possible.

Réponse technique. Rechercher l'objet par son identifiant exact, vérifier le token admin et rétablir les métadonnées sans refaire l'upload.

Vérification. Tests cv-cloudinary, cv-recovery et contrôle d'édition dans /admin.

Ce que je retiens. Cette récupération demande un identifiant correct et un accès légitime ; elle ne doit pas autoriser la consultation arbitraire de médias privés.

20 Référencement, performance et quatre types de visibilité

Un site artistique doit être lu par les humains et les moteurs

La page publique combine React et un mécanisme de pré-rendu destiné à améliorer l'indexabilité. Les informations peuvent être modifiées dynamiquement en base, mais les pages HTML produites au moment du build ne reflètent pas nécessairement instantanément le dernier contenu. Il existe donc une différence entre publier une fiche Firestore et republier une version pré-rendue à destination des moteurs de recherche.

Le code exploite des routes, des pages de détail, des métadonnées et des fichiers tels que robots.txt. La qualité SEO doit être appréciée avec le contenu réellement accessible, les titres pertinents, les descriptions, les URL canoniques et la cohérence linguistique. Je préfère mesurer les pages indexées plutôt qu'annoncer le référencement comme automatiquement garanti.

Des niveaux de visibilité à ne pas confondre

Visibilité	Public visé	Question de sécurité / SEO
Brouillon	Administrateur	Les visiteurs peuvent-ils le lire ?
Publié	Tout visiteur	Est-il présent dans les routes et métadonnées ?
Masqué	Administrateur	Les requêtes publiques l'excluent-elles ?
Fichier média public	Toute personne connaissant l'URL	La diffusion est-elle autorisée par le titulaire ?

Le bilinguisme comme modèle de contenu

Les libellés du portfolio, les données de projets et les descriptions doivent rester cohérents en français et en anglais. src/lib/content.ts prévoit un repli lorsqu'une traduction manque. Cette stratégie rend un contenu lisible plutôt que vide, mais ne remplace pas la relecture éditoriale des textes dans les deux langues. Il faut aussi vérifier que les dates, les titres, les ancres et le sens des liens restent stables entre les versions.

L'expérience mobile a des besoins propres : header fixe, navigation accessible, galerie qui ne déborde pas et boutons d'administration manipulables au toucher. Ces sujets méritent des tests exploratoires, notamment lorsque de nouvelles sections modifient la longueur du menu.

21 Publication publique, droits et maintenance du projet

Pourquoi deux dépôts sont préférables à un simple nettoyage tardif

Une version source de production peut contenir des photographies, des CV, des données de contact et d'autres informations appartenant à des tiers. Supprimer ces fichiers du dernier commit ne retire pas automatiquement leur présence des commits antérieurs. Le projet public a donc été préparé comme une édition distincte et anonymisée, avec des contenus fictifs et sans récupérer l'historique Git privé.

L'audit de publication indique le retrait de 143 images et d'un PDF dans le jeu initial. Cette mesure de confidentialité améliore la présentation académique : je peux rendre mon architecture consultable sans publier des ressources que je ne possède pas nécessairement. Un dépôt public ne donne pas non plus à un tiers une licence sur les photographies originales ; les droits sur le code et sur les médias doivent être considérés séparément.

Tenir une documentation honnête dans la durée

La CI publique exécute npm ci, la suite de tests, la vérification TypeScript et le scanner de secrets. Elle ne réalise pas encore npm run build dans le workflow principal et n'emploie pas la production Firebase ou Cloudflare pour les essais. Le dossier présente donc les tests comme des validations ciblées, pas comme une certification de l'ensemble des services. Le prochain jalon raisonnable est de renforcer l'automatisation du build, puis d'organiser des essais d'administration avec un compte non privilégié.

Pour maintenir le site, il faut sauvegarder avant de changer les règles Firestore, distinguer les migrations de contenu des déploiements du frontend et journaliser les références de médias. Ces routines ne rendent pas le projet plus spectaculaire, mais elles diminuent les erreurs qui coûtent du temps ou entraînent une perte de données.

Évolutions	Pratique recommandée	Preuve attendue
Nouveau contenu	Vérifier statut et ordre	Apparition correcte en public
Nouveau fournisseur média	Env de test et chemins de reprise	Upload puis récupération
Nouvelle règle Firestore	Sauvegarde et comparaison	Accès admin oui, tiers non
Nouvelle section navigation	Ordre ancien migré	Liens et ancres cohérents
Dépôt public	Scan historique et contenus	Aucun média privé dans les sources

22 Synthèse de méthode et décisions argumentées

Ce que je retiens du projet

La difficulté principale d'un CMS personnalisé n'est pas d'ajouter un champ dans un formulaire, mais de garantir que ce champ a le même sens dans le modèle, la sauvegarde, les permissions et la présentation publique. Ce projet m'a demandé de raisonner en systèmes : identité, éditorial, stockage, diffusion et maintenance fonctionnent ensemble, sans bénéficier d'une transaction unique.

Je peux défendre plusieurs compromis : un CMS spécifique donne un contrôle précis de l'expérience mais réclame plus de maintenance ; Firebase accélère le développement mais impose une surveillance des règles et des quotas ; Cloudinary et R2 offrent des options de stockage mais multiplient les chemins d'erreur. La présence de plusieurs choix ne signifie pas que tous sont activés dans l'environnement réel. La documentation distingue explicitement les capacités de code et la configuration de production.

Décision	Pourquoi elle a du sens	Vérification ou limite
CMS sur mesure	Édition adaptée aux tâches de l'acteur	Tests /admin et essais utilisateur
Crédits autonomes	Ne pas inventer de projet pour un rôle	Modèles et parcours d'édition
Stockage externe	Écarter les gros binaires de Firestore	Quota et configuration fournisseur
JWT vérifié côté Worker	Protéger les secrets de signature	Tests d'accès et contrôle en production
Ordre de sections migrable	Préserver les préférences historiques	Tests de compatibilité
Dépôt public nettoyé	Diffuser le code sans exposer les médias	Audit de publication

Formulation de soutenance

J'ai abordé ce projet en partant des tâches de l'utilisateur final, puis en reliant chaque tâche à un modèle de données, à une autorisation et à un mécanisme de publication. C'est cette chaîne complète que je souhaite être capable d'expliquer et de maintenir.

23 Défendre les décisions techniques à l'oral

Question possible : Pourquoi avoir créé un CMS au lieu d'un site uniquement statique ?

Réponse que je peux défendre : Parce que l'acteur doit actualiser ses projets, ses rôles, ses photos et son CV sans modifier le code. Le contenu est donc séparé de la présentation et protégé par une administration.

Question possible : Pourquoi séparer les expériences des projets ?

Réponse que je peux défendre : Un rôle professionnel peut exister sans galerie ni fiche de film. Il faut représenter les crédits comme des données autonomes plutôt que forcer un modèle inadapté.

Question possible : Pourquoi Firestore ne stocke-t-il pas toutes les photos haute résolution ?

Réponse que je peux défendre : Parce que la base a des limites de taille par document et n'est pas destinée à servir de stockage d'objets volumineux. Les binaires peuvent être envoyés vers un service média, avec leurs métadonnées en base.

Question possible : Comment le Worker reconnaît-il un véritable administrateur ?

Réponse que je peux défendre : Il vérifie cryptographiquement le JWT Firebase ainsi que l'émetteur, l'audience, les délais, le claim admin et l'e-mail vérifié ; Firestore applique indépendamment ses règles.

Question possible : Qu'est-ce qui se passe si le fichier est envoyé mais que Firestore échoue ?

Réponse que je peux défendre : Les deux services ne partagent pas une transaction. L'application signale la clé de récupération et prévoit la recherche de l'objet pour éviter des réimports aveugles.

Question possible : Pourquoi garder le dépôt original privé ?

Réponse que je peux défendre : Les anciens commits pouvaient contenir des photos et un CV. Retirer les fichiers du dernier état ne nettoie pas l'historique. J'ai donc publié une version indépendante et anonymisée.

Question possible : Une version publique qui passe les tests est-elle identique à la production ?

Réponse que je peux défendre : Non. Les contenus sont fictifs, les services tiers peuvent être désactivés et la CI ne teste pas toutes les interactions réelles. Je ne confonds pas le code démonstratif et l'environnement de production.

Question possible : Comment éviter qu'un projet masqué apparaisse au public ?

Réponse que je peux défendre : Le statut de publication est conservé dans les données et les règles Firestore restreignent la lecture des collections selon le statut et le rôle de l'utilisateur.

Question possible : Que feriez-vous ensuite ?

Réponse que je peux défendre : Je renforcerais la CI par un build Vite, puis les tests de permissions et la récupération des fichiers, avant de confirmer la facilité d'administration avec l'utilisateur final.

24 Annexes : repères de code, glossaire et limites

Fichiers à mobiliser pour une démonstration

Source	Ce que j'y vérifie
src/pages/Admin.tsx	Formulaires, CRUD, états, gestion des couvertures, médias, CV et ordre.
src/types/models.ts	Contrats TypeScript des contenus, profils, sections et références médias.
src/lib/content.ts	Souscriptions Firestore, contenus de démonstration, validation d'URLs.
src/lib/sectionOrder.ts	Fusion de l'ordre enregistré avec les sections disponibles.
src/lib/cloudinary.ts	Upload signé d'images/PDF, suppression et récupération.
src/lib/r2.ts	Upload multipart, contrôle de fichiers, reprise et indexation.
cloudinary-worker/index.js	Vérification RS256/JWKS des JWT et routes médias autorisées.
firestore.rules	Lecture publique contrôlée, claim admin, limites des médias et fallback deny.
tests/*.test.mjs	Validation admin, médias, CV, publication, navigation et langues.
AUDIT_PUBLICATION.md	Préparation du dépôt anonymisé et limites de validation.

Mini-glossaire

Notion	Explication
CMS	Système de gestion de contenu pour modifier le site sans changer le code.
CRUD	Créer, lire, modifier et supprimer une donnée.
Custom claim	Information signée dans le jeton Firebase et utilisée pour le rôle admin.
JWT / JWKS	Jeton signé et ensemble de clés publiques pour vérifier sa signature.
CORS	Politique navigateur d'autorisation des origines ; ne remplace pas l'authentification.
Upload signé	Opération autorisée et signée côté serveur pour un fournisseur média.
Multipart	Envoi d'un fichier en morceaux contrôlés et assemblés côté stockage.
Statut published	État indiquant un contenu destiné à la consultation publique.
Prérendu SEO	Production de HTML préalable pour améliorer l'indexabilité.
Dépôt anonymisé	Code de démonstration sans les médias et données réelles du propriétaire.

Documents et traçabilité

Dépôt public de référence : https://github.com/carlxwj/moussa-toure-portfolio-code . Sources internes : README.md, docs/ARCHITECTURE.md, docs/ADMIN_ET_MEDIAS.md, docs/SECURITE_DONNEES.md, docs/TESTS_ET_QUALITE.md, AUDIT_PUBLICATION.md et PUBLICATION_SANS_FUITE.md. Les guides historiques comme GUIDE_SPARK.md peuvent évoquer des architectures antérieures ; la situation déployée doit être confirmée par les configurations actives.

Cette étude de cas repose sur les fonctionnalités visibles dans les sources publiques et sur des difficultés documentées pendant les évolutions du projet. Elle n'invente pas des validations en production ni un historique personnel complet des commits. Les choix doivent pouvoir être montrés et expliqués à partir des fichiers cités.